

Zulip Follow User Feature

Handover Package

CS 5150 · Software Engineering · Spring 2026
Cornell University

Team Members	Steven Chen (sc2632) Cedric Orton-Urbina (ceo29) Alexander Wu (alw337) Gary Yan (gyy3) Jonathan Plascencia Lopez (jcp349)
Submission Date	May 15, 2026
Repository	https://github.com/Pahina0/zulip
License	Apache 2.0

Table of Contents

Table of Contents.....	1
1. Project Overview.....	2
1.1 Problem Statement.....	2
1.2 Feature Summary.....	2
1.3 Feature Demo.....	2
1.4 Scope and Deliverables.....	2
2. User's Manual.....	3
2.1 Following and Unfollowing a User.....	3
2.2 The Following Feed.....	3
2.3 Searching for Messages from Followed Users.....	3
2.4 Privacy Considerations.....	3
2.5 Push Notifications from Followed Users.....	4
3. Maintainer's Manual.....	5
3.1 Architecture Overview.....	5
3.2 Database Model.....	6
3.3 REST API Endpoints.....	6
3.4 Actions Layer.....	6
3.5 Caching.....	7
3.6 Narrow / Filter Integration.....	7
3.7 Real-Time Event Flow.....	7
3.8 Frontend Module: followed_users_data.ts.....	8
3.9 Developer Workflow.....	8
4. Test Plan and Results.....	9
4.1 Backend Tests (zerver/tests/test_followed_users.py).....	9
4.2 Frontend Tests.....	10
4.3 Known Limitations and Future Work.....	10
5. License Agreement.....	11

1. Project Overview

1.1 Problem Statement

Zulip is an open-source team communication platform used by companies and open-source communities worldwide. While Zulip allows users to follow individual topics, there was no way to follow a specific person so that their messages surfaced more prominently. High-volume workspaces suffer from signal-to-noise problems: important messages from key collaborators are buried in channel feeds.

This project adds a Follow User feature to Zulip, allowing users to designate individuals as followed and receive dedicated push notifications when those users send messages.

1.2 Feature Summary

- Follow / unfollow any user via the right-sidebar buddy list
- Following Feed — a new left-sidebar view showing only messages from followed users
- `is:followed-user` narrow filter for use in the search bar
- Backend push notification pipeline: `followed_user_push_user_ids` delivered via the existing Zulip push-notification infrastructure
- Cache-invalidation-aware storage of follow relationships in the `FollowedUser` model

1.3 Feature Demo

- Demo found at this link: https://www.youtube.com/watch?v=Hr7Z_SgvxBQ

1.4 Scope and Deliverables

Deliverable	Status
FollowedUser database model	Complete — migration applied
Follow/unfollow REST API	Complete — POST/DELETE <code>/api/v1/users/me/followed_users/{user_id}</code>
Follow button (buddy list UI)	Complete — appears on hover, toggles follow state
Following Feed sidebar view	Complete — new left-sidebar entry, <code>is:followed-user</code> narrow
Push notification pipeline	Complete (backend) — <code>enable_followed_user_push_notifications</code> flag; UI toggle pending
Real-time event propagation	Complete — <code>followed_users</code> event type, Tornado/client sync
Unit tests (backend)	In progress — test file drafted, see Section 4
Unit tests (frontend)	In progress — test file drafted, see Section 4
Help-center documentation	Stub created — requires copy-editing before merge

2. User's Manual

2.1 Following and Unfollowing a User

The follow button appears in the right sidebar (buddy list) when you hover over any user's name.

To follow a user:

- Open the right sidebar (click the people icon or press W).
- Hover over the name of the user you want to follow.
- Click the follow button (person-with-plus icon) that appears to the right of their name.
- The button changes to a filled icon confirming you are now following them.

To unfollow a user:

- Hover over the same user in the buddy list.
- Click the unfollow button (person-with-minus icon).

2.2 The Following Feed

The Following Feed is a dedicated view in the left sidebar that displays only messages sent by users you follow. It works like any other Zulip narrow — you can reply, react, and navigate as normal.

To open the Following Feed:

- Click Following feed in the left sidebar under the Views section.

The sidebar entry shows an unread count badge when followed users have sent new messages you haven't read yet.

2.3 Searching for Messages from Followed Users

You can use the search bar to filter messages from followed users in any context:

```
is:followed-user
```

This narrow can be combined with other operators:

```
is:followed-user channel:general
```

```
is:followed-user is:unread
```

The search bar will display "messages from users you follow" as the description when this filter is active.

2.4 Privacy Considerations

- Follow relationships are private: followed users are not notified that you follow them.
- The list of users you follow is not visible to other users or administrators through the UI.
- You can follow deactivated users; their past messages will still appear in your Following Feed.
- Bots can be followed the same as regular users.

2.5 Push Notifications from Followed Users

When enabled, you receive a mobile push notification whenever a user you follow sends a message that you can see. This setting is controlled by the `enable_followed_user_push_notifications` flag on your user profile.

Note: *The UI toggle for this setting has not yet been added to the Notifications settings page. Currently, the flag must be set programmatically. This is the primary remaining work item for a future contributor.*

3. Maintainer's Manual

3.1 Architecture Overview

The feature follows Zulip's standard layered architecture:

Layer	Key Files	Responsibility
Database	zerver/models/users.pyzerver/migrations/	FollowedUser model, enable_followed_user_push_notifications field
Library	zerver/lib/followed_users.py	Query helpers: get_user_follows, get_following_users, get_follow_object
Actions	zerver/actions/followed_users.py	do_follow_user, do_unfollow_user — business logic + event emission
Views (API)	zerver/views/followed_users.py	REST endpoints: follow_user, unfollow_user, get_followed_users
URL routing	zproject/urls.py	Maps /api/v1/users/me/followed_users/... to view functions
Push pipeline	zerver/actions/message_send.py	get_followers_who_can_see_stream_message — computes followed_user_push_user_ids
Caching	zerver/lib/cache.py	get_followed_users_cache_key — cache key for per-user follow sets
Frontend data	web/src/followed_users_data.ts	In-memory Set of followed user IDs; add/remove/set/initialize
Frontend filter	web/src/filter.ts	is:followed-user predicate, metadata (title, icon, description, link)
Frontend nav	web/src/navigation_views.ts	Following Feed sidebar entry, tooltip template reference
Templates	web/templates/tooltip_templates.hbsweb/templates/search_description.hbs	following-feed-tooltip-template, is:followed-user search description

3.2 Database Model

FollowedUser (zerver/models/users.py)

Field	Type	Description
user_profile	FK(UserProfile)	The user who is doing the following
followed_user	FK(UserProfile)	The user being followed
date_followed	DateTimeField	Timestamp when the follow was created (auto_now_add)

enable_followed_user_push_notifications (UserProfile)

A BooleanField (default=False) added to the UserProfile model. When True for a user, they receive push notifications whenever a user they follow sends a message visible to them.

3.3 REST API Endpoints

Method	URL	Auth	Description
GET	/api/v1/users/me/followed_users	Authenticated user	Returns list of followed user IDs with timestamps
POST	/api/v1/users/me/followed_users/{user_id}	Authenticated user	Follow user_id
DELETE	/api/v1/users/me/followed_users/{user_id}	Authenticated user	Unfollow user_id

Error responses:

- 400 You cannot follow yourself.
- 400 User is already followed. (POST to already-followed user)
- 400 User is not followed. (DELETE with no prior follow)

3.4 Actions Layer

do_follow_user(user_profile, target_user)

- Creates a FollowedUser row.
- Invalidates the get_followed_users_cache_key for target_user.
- Emits a followed_users Tornado event so all of the follower's clients update immediately.

do_unfollow_user(user_profile, target_user)

- Idempotent: no-ops (but still emits an event) if the relationship doesn't exist.
- Deletes the FollowedUser row.
- Invalidates cache and emits event as above.

3.5 Caching

`get_followed_users_cache_key(user_id)` (`zerver/lib/cache.py`, line ~559) stores the set of user IDs who follow a given user. It is populated lazily by `get_following_users(user_id)` and invalidated on every follow or unfollow action. Tests must account for this: assert the cache key is `None` after an action and repopulated after the next call.

3.6 Narrow / Filter Integration

The `is:followed-user` narrow is handled in `web/src/filter.ts`:

- Predicate: checks `followed_users_data.is_user_followed(message.sender_id)`.
- Metadata: `title = "Following feed"`, `icon = "follow"`, `description = "Messages from users you follow."`, `link = "/help/follow-a-user"`.
- `can_newly_match_moved_messages`: returns `true` (moved messages from followed users can appear).

The search bar description ("messages from users you follow") is rendered in `web/templates/search_description.hbs` via the `is_operator` block.

3.7 Real-Time Event Flow

When a user follows or unfollows someone:

- The action layer calls `send_event` with type: `"followed_users"` and the updated `followed_users` list.
- Tornado pushes the event to all active clients for that user.
- The client-side event handler calls `followed_users_data.set_followed_users(event.followed_users)`, keeping the in-memory set in sync.

When a message is sent:

- `get_followers_who_can_see_stream_message(realm_id, sender_id, stream_id, topic_name)` computes the set of users who (a) follow the sender, (b) have `enable_followed_user_push_notifications=True`, and (c) can see the message.
- These user IDs are passed into the push notification queue as `followed_user_push_user_ids`.

3.8 Frontend Module: followed_users_data.ts

```
// web/src/followed_users_data.ts
const followed_users = new Set<number>();
```

Public API:

Function	Description
initialize(data)	Called on page load with the server-provided followed_users array
set_followed_users(list)	Replaces the entire set — used on real-time events
add_followed_user(id)	Adds a single user ID
remove_followed_user(id)	Removes a single user ID (idempotent)
is_user_followed(id)	Returns true if the ID is in the set

3.9 Developer Workflow

Setup

```
git clone https://github.com/zulip/zulip
cd zulip && ./tools/provision
vagrant up && vagrant ssh
./tools/run-dev    # dev server at http://localhost:9991
```

Running Tests

```
./tools/test-backend zerver/tests/test_followed_users.py
./tools/test-js-with-node web/tests/followed_users_data.test.cjs
./tools/test-js-with-node web/tests/filter.test.cjs
./tools/lint zerver/tests/test_followed_users.py
```

4. Test Plan and Results

4.1 Backend Tests (zerver/tests/test_followed_users.py)

The backend test class `FollowedUsersTests(ZulipTestCase)` covers the full API surface. Hamlet is the follower; Cordelia is the followed user — mirroring the convention used in `test_muted_users.py`.

Test Method	What It Verifies	Status
<code>test_get_user_follows</code>	GET endpoint returns empty list, then correct entry after follow	Passing
<code>test_follow_self</code>	Following yourself returns 400 with appropriate error	Passing
<code>test_follow_bot</code>	Bots can be followed and unfollowed	Passing
<code>test_follow_already_followed</code>	Duplicate follow returns 400	Passing
<code>test_follow_valid_data</code>	Follow succeeds, DB row created, timestamp correct	Passing
<code>test_follow_deactivated_user</code>	Deactivated users can still be followed	Passing
<code>test_unfollow_before_following</code>	Unfollow without follow returns 400	Passing
<code>test_unfollow_valid_data</code>	Unfollow removes DB row, <code>get_user_follows</code> updates	Passing
<code>test_unfollow_deactivated_user</code>	Deactivated users can be unfollowed	Passing
<code>test_get_following_users_cache</code>	Cache populated then invalidated on follow/unfollow	Passing
<code>test_follow_sends_event</code>	Follow action emits <code>followed_users</code> event to correct users	Passing
<code>test_unfollow_sends_event</code>	Unfollow action emits event with empty <code>followed_users</code> list	Passing
<code>test_unfollow_is_idempotent_at_action_layer</code>	<code>do_unfollow_user</code> does not raise on non-existent follow	Passing

4.2 Frontend Tests

`web/tests/followed_users_data.test.cjs`

Test	What It Verifies	Status
<code>edge_cases</code>	<code>is_user_followed(undefined)</code> and <code>(0)</code> return false	Passing
<code>add_and_remove_follows</code>	<code>add/remove</code> are idempotent; <code>is_user_followed</code> reflects state	Passing
<code>set_followed_users</code>	Bulk set replaces state; empty set clears all	Passing
<code>initialize</code>	<code>initialize()</code> populates correct IDs from server payload	Passing

`web/tests/filter.test.cjs` additions

- Predicate test: `is:followed-user` returns `true/false` based on `followed_users_data.is_user_followed` mock.
- Metadata test: correct title, icon, description, redirect URL, and help link for `is:followed-user`.
- `can_newly_match_moved_messages`: returns `true` for `is:followed-user` and negated variant.

4.3 Known Limitations and Future Work

Limitation	Recommended Next Step
No UI toggle for <code>enable_followed_user_push_notifications</code>	Add a checkbox to Settings > Notifications
No ability to view who you are following from profile page	Add "Following" tab to user settings
Help center article is a stub	Write full MDX article in <code>starlight_help/src/content/docs/</code>
API changelog entry uses placeholder feature level	Run <code>tools/create-api-changelog</code> to finalize before merge

5. License Agreement

All code contributed as part of this project is submitted to the Zulip open-source project under the Apache License, Version 2.0. This is consistent with Zulip's existing license and with Cornell University's open-source contribution policy for CS 5150 course projects.

License Apache License, Version 2.0 SPDX Identifier Apache-2.0 Full text https://www.apache.org/licenses/LICENSE-2.0	Permissions ✓ Commercial use ✓ Modification ✓ Distribution ✓ Patent use ✓ Private use Conditions → License and copyright notice required → State changes must be documented
--	--

By contributing to this project, all team members agree to release their contributions under the Apache 2.0 license.

Cedric Orton-Urbina (ceo29)	Date: May 14, 2026
Steven Chen (sc2632)	Date: May 14, 2026
Alexander Wu (alw337)	Date: May 14, 2026
Gary Yan (gyy3)	Date: May 14, 2026
Jonathan Plascencia Lopez (jcp349)	Date: May 14, 2026

Signatures